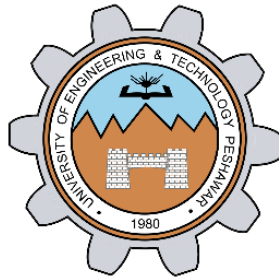


Object Oriented Programming Lab

Object Programming Essentials

LAB # 01



Fall 2025

CSE208L Object Oriented Programming Lab

Submitted by: **Muhammad Mohsin Khan**

Registration No.: **24PWCSE2382**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Student Signature: _____

Submitted to:

Engr. Sumayya Salahuddin

September 25, 2025

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Object Oriented Programming Lab

Lab Objectives: Exploring OOP Basics

1. Learn OOP Benefits: Understand why OOP is valuable.
2. Grasp Classes and Objects: Get to know classes and objects.
3. Build a Basic Class: Create a class with data and functions.
4. Manage Access: Use access specifiers to control class members.
5. Create Constructors: Make simple and overloaded constructors.
6. Work with Objects: Use objects and functions to handle data.
7. Practice Skills: Gain experience with classes and objects.

Activity #1.4.1

Title: Build the LightBulb Class

Problem Breakdown:

Design a LightBulb class to simulate a light bulb with these features:

1. Key Property: Add a Boolean isOn to show if the bulb is on or off.
2. Default Start: Include a constructor that sets isOn to false.
3. Control Methods:
 - turnOn(): Set isOn to true.
 - turnOff(): Set isOn to false.
4. Check Status: Create a getState() method (const) to return the bulb's state.
5. Display Status: Add a show() method (const) to show "The light bulb is now ON" or "OFF" using getState().
6. Test It: In main(), create a LightBulb object, use turnOn() and turnOff(), and display its state with show().

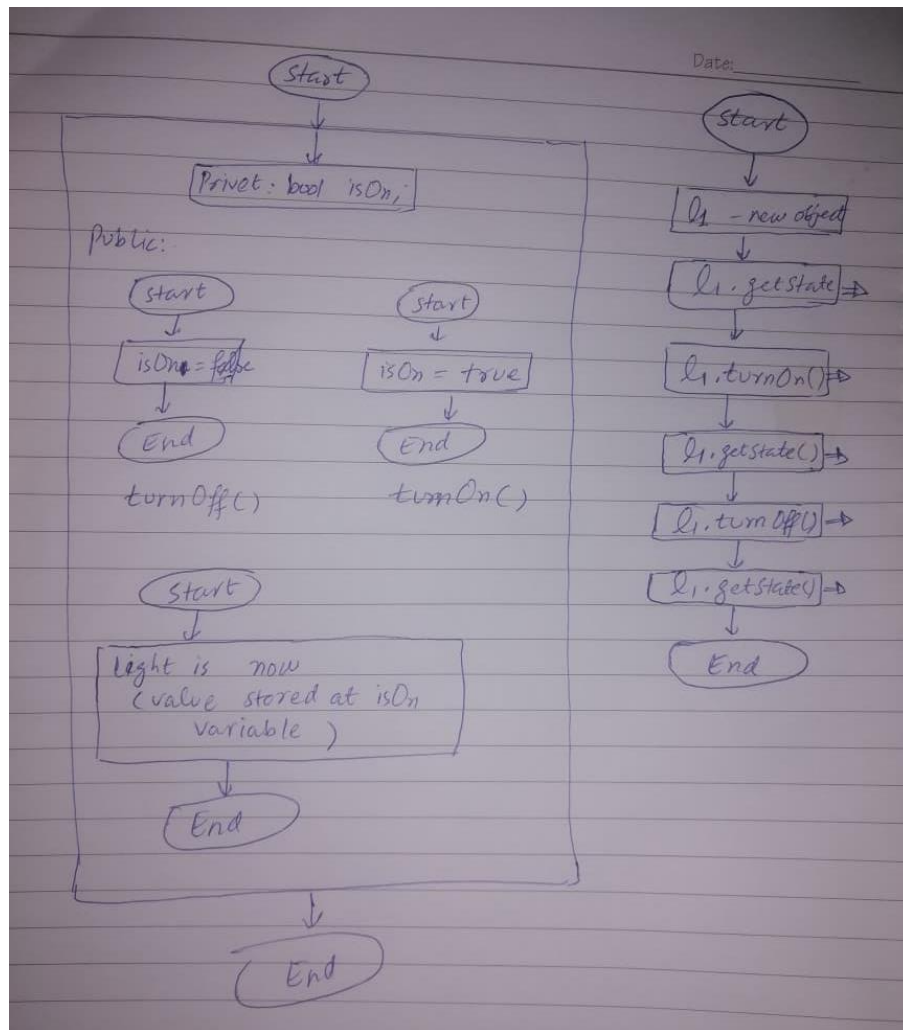
Algorithm:

UML diagram for this program is given below.

- Make a class called LightBulb.
- Declare isOn as a private boolean data member.
- Declare a no argument constructor that sets the value of isOn to false.
- Define turnOn and turnOff methods to set the value of isOn to true and false respectively.
- Define a getState method that returns the current value of isOn.
- Define the show method that calls getState and displays the corresponding message
- In main() , create an object of the LightBulb to demonstrate the use of LightBulb.

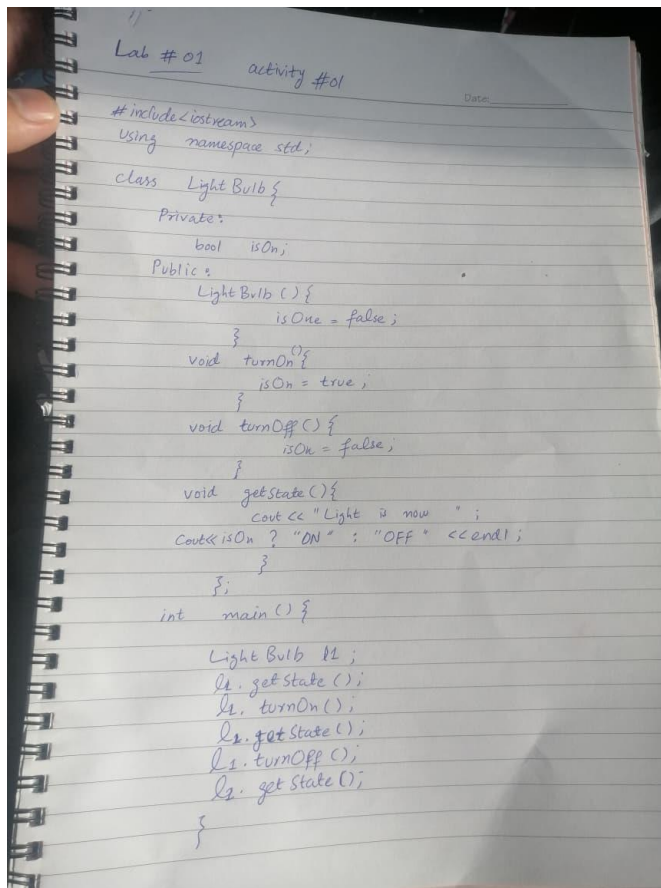
Object Oriented Programming Lab

Flowchart:



Object Oriented Programming Lab

Hand-written Code:



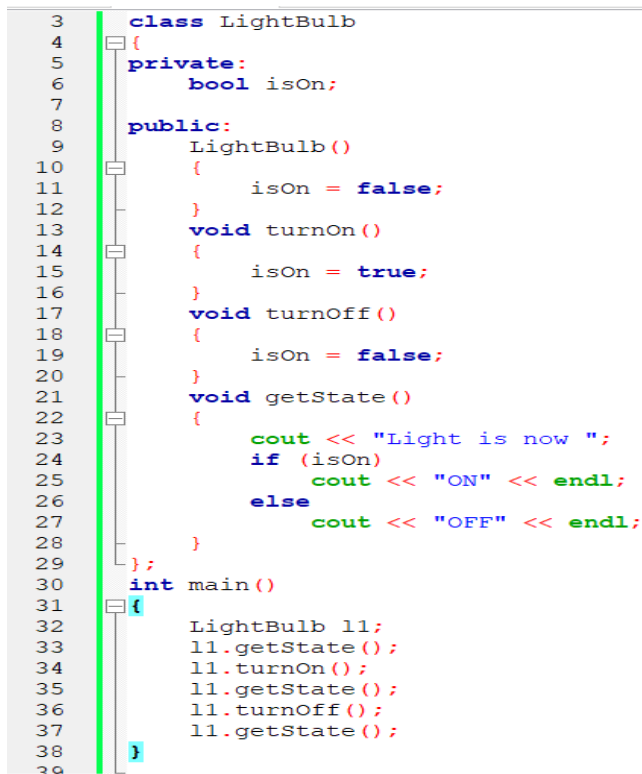
Lab # 01 activity #01

```
#include <iostream>
using namespace std;

class LightBulb {
private:
    bool isOn;
public:
    LightBulb() {
        isOn = false;
    }
    void turnOn() {
        isOn = true;
    }
    void turnOff() {
        isOn = false;
    }
    void getState() {
        cout << "Light is now ";
        cout << isOn ? "ON" : "OFF" << endl;
    }
};

int main() {
    LightBulb l1;
    l1.getState();
    l1.turnOn();
    l1.getState();
    l1.turnOff();
    l1.getState();
}
```

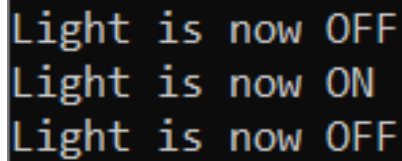
Source-Code:



```
3  class LightBulb
4  {
5      private:
6          bool isOn;
7
8      public:
9          LightBulb()
10         {
11             isOn = false;
12         }
13         void turnOn()
14         {
15             isOn = true;
16         }
17         void turnOff()
18         {
19             isOn = false;
20         }
21         void getState()
22         {
23             cout << "Light is now ";
24             if (isOn)
25                 cout << "ON" << endl;
26             else
27                 cout << "OFF" << endl;
28         }
29     };
30     int main()
31     {
32         LightBulb l1;
33         l1.getState();
34         l1.turnOn();
35         l1.getState();
36         l1.turnOff();
37         l1.getState();
38     }
39 }
```

Object Oriented Programming Lab

Output:



```
Light is now OFF
Light is now ON
Light is now OFF
```

Activity # 1.4.2

Title

Create a class named **GPSCoordinates** that represents latitude and longitude for a location.

Problem Analysis

We need to design a class that stores the geographical position of a place using latitude and longitude. These two values represent the exact position of a location on Earth.

The class should have two main data members:

- latitude → stores the latitude value.
- longitude → stores the longitude value.

To make the class flexible, we will define two constructors:

1. A no-argument constructor that sets both latitude and longitude to 0.
2. A two-argument constructor that accepts values for latitude and longitude and assigns them to the data members.

Demonstration in main()

In the main() function, we will:

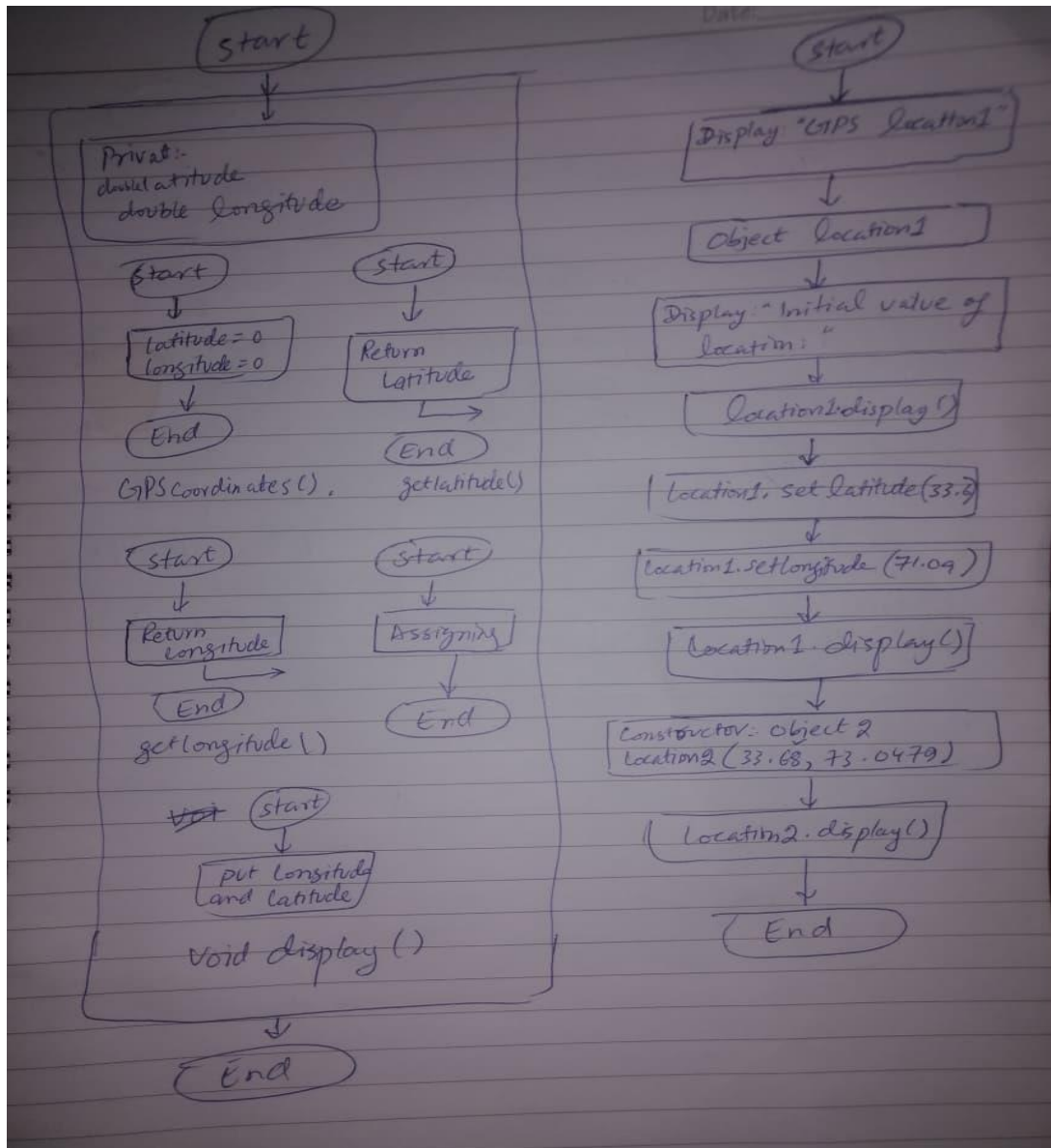
- Create an object using the **default constructor** and show its initial values.
- Update its latitude and longitude using the setter methods and display the new values.
- Create another object using the **two-argument constructor** and display its values directly.

Algorithm (Step-by-Step)

1. Define a class called GPSCoordinates.
2. Declare latitude and longitude as private data members.
3. Create a no-argument constructor that sets both values to 0.
4. Create a two-argument constructor that assigns values to latitude and longitude.
5. Define setLatitude() and setLongitude() to update the values.
6. Define getLatitude() and getLongitude() to return the values (both const).
7. Define display() to print the coordinates in (latitude, longitude) format (const).
8. In main(), create objects using both constructors, update values using setters, and print them using display().

Object Oriented Programming Lab

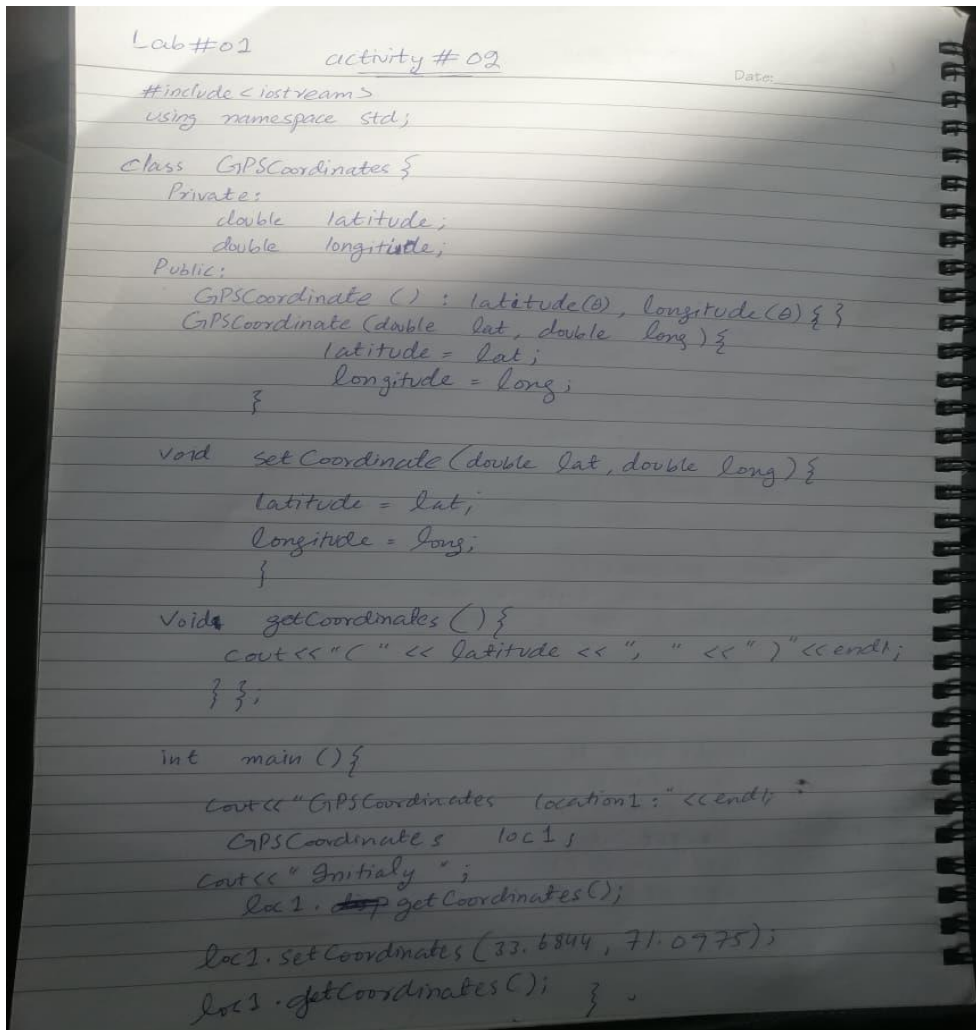
FlowChart:



Object Oriented Programming Lab

Hand-written Code:

Source Code:



Lab #01 activity #02 Date: _____

```
#include <iostream>
using namespace std;

class GPSCoordinates {
    Private:
        double latitude;
        double longitude;
    Public:
        GPSCoordinate() : latitude(0), longitude(0) {}
        GPSCoordinate(double lat, double long) {
            latitude = lat;
            longitude = long;
        }

        void setCoordinate(double lat, double long) {
            latitude = lat;
            longitude = long;
        }

        void getCoordinates() {
            cout << "(" << latitude << ", " << longitude << ")" << endl;
        }
};

int main() {
    cout << "GPSCoordinates location1:" << endl;
    GPSCoordinates loc1;
    cout << "Initially ";
    loc1.getCoordinates();

    loc1.setCoordinates(33.6844, 71.0775);
    loc1.getCoordinates();
}
```

Object Oriented Programming Lab

```
9
10 public:
11     GPSCoordinates() : latitude(0), longitude(0) {}
12
13     GPSCoordinates(double lat, double lon) : latitude(lat), longitude(lon) {}
14
15     double getLatitude() const
16     {
17         return latitude;
18     }
19
20     double getLongitude() const
21     {
22         return longitude;
23     }
24
25     void setLatitude(double lat)
26     {
27         latitude = lat;
28     }
29
30     void setLongitude(double lon)
31     {
32         longitude = lon;
33     }
34
35     void display() const
36     {
37         cout << "(" << latitude << ", " << longitude << ")" << endl;
38     }
39 };
40
41 int main()
42 {
43     cout << "GPSCoordinates location1:" << endl;
44     GPSCoordinates location1;
45     cout << "Initial value of location1: ";
46     location1.display();
47
48     location1.setLatitude(33.6844);
49     location1.setLongitude(71.0975);
50     cout << "Updated value of location1: ";
51     location1.display();
52
53     cout << "\nGPSCoordinates location2:" << endl;
54     GPSCoordinates location2(33.6844, 73.0479);
```

Output:

```
GPSCoordinates location1:
Initial value of location1: (0, 0)
Updated value of location1: (33.6844, 71.0975)
```

Conclusion:

This program provides a clear understanding of the fundamental concepts of classes and objects in programming. It serves as a foundation for learning object-oriented programming (OOP) and prepares us for solving more complex problems in the future. Through this program, we gain knowledge about constructors and methods, which are essential building blocks of OOP.

Object Oriented Programming Lab

Attach Lab Rubrics Here: